

1. 100. Same Tree

```
def isSameTree(p, q):  
    if not p and not q: return True  
    if not p or not q or p.val != q.val: return False  
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

2. 104. Maximum Depth of Binary Tree

```
def maxDepth(root):  
    if not root: return 0  
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

3. 101. Symmetric Tree

```
def isSymmetric(root):  
    def isMirror(t1, t2):  
        if not t1 and not t2: return True  
        if not t1 or not t2 or t1.val != t2.val: return False  
        return isMirror(t1.left, t2.right) and isMirror(t1.right, t2.left)  
    return isMirror(root, root)
```

4. 110. Balanced Binary Tree

```
def isBalanced(root):  
    def check(node):  
        if not node: return 0  
        left = check(node.left)  
        right = check(node.right)  
        if left == -1 or right == -1 or abs(left - right) > 1: return -1  
        return 1 + max(left, right)  
    return check(root) != -1
```

5. 226. Invert Binary Tree

```
def invertTree(root):  
    if root:
```

```
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

6. 111. Minimum Depth of Binary Tree

```
def minDepth(root):
    if not root: return 0
    if not root.left: return 1 + minDepth(root.right)
    if not root.right: return 1 + minDepth(root.left)
    return 1 + min(minDepth(root.left), minDepth(root.right))
```

7. 112. Path Sum

```
def hasPathSum(root, targetSum):
    if not root: return False
    if not root.left and not root.right:
        return root.val == targetSum
    return hasPathSum(root.left, targetSum - root.val) or hasPathSum(root.right, targetSum - r
```

8. 617. Merge Two Binary Trees

```
def mergeTrees(t1, t2):
    if not t1 and not t2: return None
    if not t1: return t2
    if not t2: return t1
    t1.val += t2.val
    t1.left = mergeTrees(t1.left, t2.left)
    t1.right = mergeTrees(t1.right, t2.right)
    return t1
```

9. 404. Sum of Left Leaves

```
def sumOfLeftLeaves(root):
    if not root: return 0
    res = 0
    if root.left and not root.left.left and not root.left.right:
        res += root.left.val
    return res + sumOfLeftLeaves(root.left) + sumOfLeftLeaves(root.right)
```

10. 543. Diameter of Binary Tree

```
def diameterOfBinaryTree(root):  
    diameter = 0  
    def dfs(node):  
        nonlocal diameter  
        if not node: return 0  
        left = dfs(node.left)  
        right = dfs(node.right)  
        diameter = max(diameter, left + right)  
        return 1 + max(left, right)  
    dfs(root)  
    return diameter
```